SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1741**

CO1-AT2-Constraint-Based Problem Solving

Register Number	**192312174**
Name	**Pavithra S**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS:3 **Total Marks:30**

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.
   The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

   Constraints:

   - The robot can move up, down, left, right

- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:
a) Analyze all the given constraints and explain how they affect the robot's decision-making process.
b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

Code of Conduct:

I ____Pavithra S(192312174)_____ certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Problem Understanding	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	6
Constraint Analysis	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	6
Solution Development	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	7.5

Application of Concepts	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	6
Justification	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	4.5

Question 1 – Assigning Doctors to Shifts (Backtracking Search)

1. Problem Understanding

A hospital has 3 doctors (D1, D2, D3) and 3 shifts (Morning, Afternoon, Night). We must give each doctor one shift so that all rules are followed. This is a Constraint Satisfaction Problem (CSP).

Part of a CSP	In this problem
Variables	D1, D2, D3 (the doctors)
Domain (possible values)	Morning (M), Afternoon (A), Night (N) for each doctor
Order of shifts	Morning < Afternoon < Night
Goal	Give every doctor a shift so that all 5 constraints are true

The five constraints given in the question:

No.	Constraint	Short name
C1	D1 cannot work the Night shift	D1 ≠ N
C2	D2 must work before D3	shift(D2) < shift(D3)
C3	Only one doctor per shift	All shifts different
C4	D3 cannot work the Morning shift	D3 ≠ M
C5	Every doctor gets exactly one shift	Each variable gets one value

2. Constraint Analysis

- **Single-doctor constraints (C1, C4):** these only look at one doctor, so they are the easiest to check. They cut the choices down at once: D1 can only be M or A, and D3 can only be A or N.
- **All-different constraint (C3):** once a shift is given to one doctor, no other doctor can take it.
- **Order constraint (C2):** this connects two doctors, so it can only be checked after both D2 and D3 have shifts.
- **Checking order:** check the easy constraints (C1, C4, C3) as soon as a doctor is assigned, and check C2 when D3 is assigned. Checking early lets us reject bad choices quickly.

3. Solution Development – Backtracking Steps

Method: assign doctors in the order D1 → D2 → D3. For each doctor, try the shifts in the order M, A, N. After every try, check the constraints. If a constraint fails, reject that value and try the next one. If no value works, go back (backtrack) to the previous doctor.

Step	Try	Partial schedule	Constraint check	Result
1	D1 = Morning	D1:M	C1 ✓ (not Night); shift free ✓	Accept
2	D2 = Morning	D1:M, D2:M	C3 ✗ – Morning already used by D1	Reject
3	D2 = Afternoon	D1:M, D2:A	C3 ✓ shift free	Accept
4	D3 = Morning	D1:M, D2:A, D3:M	C3 ✗ (Morning used) and C4 ✗ (D3 cannot work Morning)	Reject
5	D3 = Afternoon	D1:M, D2:A, D3:A	C3 ✗ – Afternoon already used by D2	Reject
6	D3 = Night	D1:M, D2:A, D3:N	C3 ✓, C4 ✓, C2 ✓ (Afternoon is before Night)	Accept – all assigned

Final check of all constraints for the schedule D1 = Morning, D2 = Afternoon, D3 = Night:

No.	Check	Result
C1	D1 works Morning, not Night	✓ Satisfied
C2	D2 (Afternoon) is before D3 (Night)	✓ Satisfied
C3	Morning, Afternoon and Night each have one doctor	✓ Satisfied
C4	D3 works Night, not Morning	✓ Satisfied
C5	D1, D2, D3 each have exactly one shift	✓ Satisfied

Final valid schedule
Morning → D1 Afternoon → D2 Night → D3

Extra: seeing real backtracking. To show how the search goes back, we keep searching the other branches:

Branch	What happens	Result
D1 = M, D2 = N	Only Afternoon is left for D3. C2 fails because Night is not before Afternoon.	Backtrack
D1 = A, D2 = M	D3 gets Night. C1, C2, C3, C4 all pass.	Second valid schedule: D1-A, D2-M, D3-N
D1 = A, D2 = N	Only Morning is left for D3, but C4 says D3 cannot work Morning.	Backtrack
D1 = N	C1 fails straight away.	Rejected

4. Application of Concepts

- **Constraint Satisfaction Problem:** variables = doctors, domain = shifts, constraints = the five hospital rules.
- **Backtracking search:** this is depth-first search that assigns one variable at a time and checks constraints after each step. It stops a path as soon as a constraint fails.
- **Constraint checking (consistency):** checking constraints early avoids exploring many bad schedules.

5. Justification

- The schedule passes all five constraints (see the final check table), so it is valid.
- Without checking, there are $3 \times 3 \times 3 = 27$ ways to give shifts. Backtracking rejected wrong choices early (steps 2, 4 and 5) and found a solution in only 6 steps.
- Backtracking is a good choice here because the problem is small and every rule can be checked as soon as the values it needs are assigned.

Question 2 – Robot in a 5×5 Grid (BFS and Priority Queue)

1. Problem Understanding

A robot must travel from Start (S) to Goal (G) in a 5×5 grid. It can move Up, Down, Left or Right. Every move costs 1. It cannot pass through obstacles. We must find the best (lowest cost) path.

Assumption: the question does not include the grid picture, so the grid below is assumed. Cells are written as (row, column), starting from (0,0) at the top-left. If your question paper has a different grid, only the cell positions change – the method and table format stay the same.

Note on the question: it names BFS but also mentions a priority queue and the Manhattan distance. BFS itself uses a normal (FIFO) queue. A priority queue ordered by cost + Manhattan distance is the A* method. To cover both, Part A uses BFS with a queue and Part B uses a priority queue with the Manhattan heuristic.

	c0	**c1**	**c2**	**c3**	**c4**
r0	**S**				
r1					
r2					
r3					
r4					**G**

S = Start (0,0) G = Goal (4,4) X = Obstacles at (0,3), (1,1), (1,3), (2,1), (3,3)

2. Constraint Analysis

Constraint	How it affects the search
Moves: Up, Down, Left, Right	Each cell has at most 4 neighbours. No diagonal moves.
Cost of each move = 1	All steps cost the same, so a path with fewer moves is always cheaper.

Constraint	How it affects the search
Cannot pass obstacles	Obstacle cells are never added to the queue. This is checked first for every neighbour.
Manhattan distance heuristic	h(n) = |row − 4| + |col − 4|. It is the minimum number of moves to the goal, so it never over-estimates (it is admissible). It guides the priority queue.

Order of checking: (1) is the neighbour inside the grid? (2) is it an obstacle? (3) has it been visited already? Only then is it added to the queue.

3. Solution Development

Part A – BFS (FIFO queue)

BFS expands the oldest cell in the queue first. Neighbours are added in the order Up, Down, Left, Right. The search stops when G is added to the queue.

Step	Expand	New cells added	Queue after this step
1	(0,0)	(1,0), (0,1)	(1,0) (0,1)
2	(1,0)	(2,0)	(0,1) (2,0)
3	(0,1)	(0,2)	(2,0) (0,2)
4	(2,0)	(3,0)	(0,2) (3,0)
5	(0,2)	(1,2)	(3,0) (1,2)
6	(3,0)	(4,0), (3,1)	(1,2) (4,0) (3,1)
7	(1,2)	(2,2)	(4,0) (3,1) (2,2)
8	(4,0)	(4,1)	(3,1) (2,2) (4,1)
9	(3,1)	(3,2)	(2,2) (4,1) (3,2)
10	(2,2)	(2,3)	(4,1) (3,2) (2,3)
11	(4,1)	(4,2)	(3,2) (2,3) (4,2)
12	(3,2)	none	(2,3) (4,2)
13	(2,3)	(2,4)	(4,2) (2,4)
14	(4,2)	(4,3)	(2,4) (4,3)
15	(2,4)	(1,4), (3,4)	(4,3) (1,4) (3,4)
16	(4,3)	(4,4)	(1,4) (3,4) (4,4)

At step 16, the goal (4,4) is added to the queue, so BFS stops. BFS expanded 16 cells.

Part B – Priority queue with Manhattan distance (f = g + h)

Here g = cost from S so far, h = Manhattan distance to G, f = g + h. The cell with the smallest f is expanded first. If f is equal, the cell with the smaller h goes first.

Step	Expand	g, h, f	New cells added to priority queue	Priority queue after this step (cell : f)
1	(0,0)	g=0, h=8, f=8	(1,0) g=1,h=7,f=8; (0,1) g=1,h=7,f=8	(1,0):8 (0,1):8
2	(1,0)	g=1, h=7, f=8	(2,0) g=2,h=6,f=8	(2,0):8 (0,1):8
3	(2,0)	g=2, h=6, f=8	(3,0) g=3,h=5,f=8	(3,0):8 (0,1):8
4	(3,0)	g=3, h=5, f=8	(4,0) g=4,h=4,f=8; (3,1) g=4,h=4,f=8	(4,0):8 (3,1):8 (0,1):8
5	(4,0)	g=4, h=4, f=8	(4,1) g=5,h=3,f=8	(4,1):8 (3,1):8 (0,1):8
6	(4,1)	g=5, h=3, f=8	(4,2) g=6,h=2,f=8	(4,2):8 (3,1):8 (0,1):8
7	(4,2)	g=6, h=2, f=8	(3,2) g=7,h=3,f=10; (4,3) g=7,h=1,f=8	(4,3):8 (3,1):8 (0,1):8 (3,2):10
8	(4,3)	g=7, h=1, f=8	(4,4) g=8,h=0,f=8	(4,4):8 (3,1):8 (0,1):8 (3,2):10
9	(4,4)	g=8, h=0, f=8	Goal reached	-

The goal is taken out of the priority queue at step 9, so the search ends. Only 9 cells were expanded.

Path found (follow the parent of each cell back from G):

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)
Moves: Down, Down, Down, Down, Right, Right, Right, Right
Number of moves = 8 Total cost = 8 × 1 = 8

Other paths of cost 8 also exist (for example through (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4)). No path can be cheaper than 8, because the Manhattan distance from S to G is already 8.

4. Application of Concepts

Concept	How it is used here
BFS (uninformed search)	Expands cells level by level using a FIFO queue. It uses no knowledge about where the goal is. Because every move costs 1, the first time BFS reaches G it has found the cheapest path.
Priority queue	Always gives the cell with the lowest f = g + h. This sends the search towards the goal.
Manhattan distance heuristic	Gives a quick, never-too-high estimate of the remaining cost to G.
Optimality and completeness	Both BFS and the priority-queue method find a path if one exists, and both return a path of the lowest cost (8).

5. Justification

- Both methods give the same answer: cost 8. This agrees with the Manhattan lower bound of 8, so the path is proven optimal.

Point	BFS (queue)	Priority queue (f = g + h)
Cells expanded	16	9
Uses knowledge of the goal?	No	Yes (Manhattan distance)
Path cost	8 (optimal)	8 (optimal)

- The priority-queue method expanded fewer cells because the heuristic pushed the search towards G, while BFS spread out in all directions.
- Obstacles were never entered, so all the constraints of the question were followed.

Question 3 – Autonomous Rescue Robot (Online Search)

1. Problem Understanding [Task (a), first part]

A rescue robot is inside a damaged building made of rooms (a grid). It starts at S and must reach a survivor at G. The robot cannot see the whole building. It can only see the cells next to it. So it must learn the map while it moves.

No.	Constraint given	Simple meaning
1	Moves: up, down, left, right	The robot moves one cell at a time in 4 directions.
2	Some cells are blocked	The robot can never enter these cells.
3	Can only sense adjacent cells	The robot does not know the whole map in advance.
4	Move cost = 1; risky zone adds 2	A normal move costs 1. Entering a risky cell costs 1 + 2 = 3.
5	Minimise cost and stay safe	Find the cheapest route, but avoid danger.
6	Update knowledge dynamically	The robot changes its map and plan as it discovers new things (online search).

2. Constraint Analysis [Task (a), second part]

How each constraint affects the robot's decisions:

Constraint	Effect on decision-making
Blocked cells	Cells known to be blocked are removed from the possible moves. If the robot finds a new blocked cell, its old plan may stop working and it must make a new plan.
Limited sensing	The robot cannot plan the full route at the start. It must plan with what it knows, move, look again, and then update the plan.
Risky zones (extra cost 2)	A short route through risky cells may cost more than a longer safe route. The robot compares total costs and does not simply choose the shortest path.
Cost minimisation	The robot must compare routes by total cost (sum of step costs), not by the number of steps.

Constraint	Effect on decision-making
Safe navigation	Blocked cells are never entered. Risky cells are entered only when the saving is worth it.
Dynamic knowledge update	The robot stores a map in memory (unknown / free / blocked / risky) and corrects it after every sensor reading.

Priority of the constraints (what the robot checks first):

1. Blocked cells – never allowed (hard constraint).
2. Safety and risky zones – avoid unless the cost saving is clear.
3. Total cost – choose the cheapest of the allowed routes.
4. Limited sensing – update the map and plan again after every move.

Small example: Route 1 has 4 moves, with 2 of them into risky cells: cost = 4 + (2 × 2) = 8. Route 2 is safe but longer, with 6 moves: cost = 6. The robot chooses Route 2, because its cost is lower and it is safer.

3. Solution Development [Task (b)]

Chosen approach: Uniform Cost Search (UCS) used inside an Online Search Agent. UCS finds the cheapest path on the map the robot knows now. The online agent repeats this after every move because the map keeps changing.

Steps the robot follows:

Step	What the robot does	Explanation
1	Start and sense	Stand at S. Look at the 4 adjacent cells. Record each one as free, blocked or risky in the map.
2	Plan with UCS	Run UCS on the known map. g(n) = total cost so far (1 per normal move, 3 per risky move). Always expand the cell with the lowest g. Cells not yet seen are treated as free with cost 1 (an optimistic guess).
3	Take one step	Move one cell along the cheapest planned path.
4	Sense again	Look at the new adjacent cells and update the map.
5	Check the plan	If a planned cell is now blocked, or a risky cell appears, the old plan is no longer good. Make a new plan from the current cell (go back to step 2). Otherwise continue.

Step	What the robot does	Explanation
6	Handle dead ends	If no known route reaches G, go to the nearest unexplored cell and explore from there. Remember visited cells so the robot does not loop.
7	Repeat	Repeat steps 2–6 until the robot reaches G. Then it starts the rescue.

In short (pseudo-code):

```
map = {start}
while position != G:
    sense adjacent cells -> update map
    path = UniformCostSearch(map, position, G)
    if no path: move to nearest unexplored cell
    else: move one step along path
```

4. Application of Concepts [Task (c)]

(i) Intelligent agent

An intelligent agent senses its environment with sensors and acts on it with actuators. Our robot is a goal-based and utility-based agent with memory (a model-based agent): it keeps a map, has a goal (reach the survivor), and prefers lower cost and higher safety.

PEAS part	Rescue robot
Performance measure	Reach the survivor, lowest total cost, avoid risky zones, stay safe
Environment	Damaged building (grid of rooms) with blocked and risky cells
Actuators	Wheels / legs for moving Up, Down, Left, Right
Sensors	Sensors that see the 4 adjacent cells; position sensor

(ii) Rationality

A rational agent chooses the action that is expected to give the best result, using the information it has so far. Our robot is rational because it:

- compares the total cost of routes (not just the number of steps);
- avoids blocked cells and treats risky cells carefully;

- changes its plan when new information arrives, instead of following an old plan blindly.

The robot is not expected to know everything. It is rational if it makes the best choice with what it has sensed so far.

(iii) Environment type

Property	Type	Reason
Observability	Partially observable	The robot only sees adjacent cells.
Number of agents	Single agent	Only one robot is working.
Determinism	Deterministic (assumed)	A move command results in the intended move.
Episodic or sequential	Sequential	Each move changes the position and affects later choices and cost.
Static or dynamic	Mostly static	The building layout stays the same. Only the robot's knowledge changes. (It becomes dynamic if debris keeps moving.)
Discrete or continuous	Discrete	The building is a grid of separate cells.
Known or unknown	Unknown at start	The robot learns the map by exploring.

5. Justification [Task (d)]

Why Uniform Cost Search inside an Online Search Agent is the most suitable approach:

- **Costs are not equal.** Because risky cells cost 3 and normal cells cost 1, BFS (which counts steps) can give a wrong answer. UCS adds up real costs, so it always finds the cheapest path on the known map.
- **Partial observation needs an online method.** An offline planner needs the full map in advance. The online agent plans with what it knows and fixes the plan when it learns more.
- **Safety.** Since risky cells have a higher cost, UCS avoids them unless they save a lot of cost. Blocked cells are never used.
- **Works with limited memory and sensing.** The robot only needs its current map and position to plan again.

- **Complete and optimal.** UCS always finds a path if one exists, and it is the cheapest one on the current map, because all step costs are positive.

Other method	Problem here	Why UCS is better
BFS	Ignores the extra cost of risky cells	UCS uses real costs
DFS	Can take a very long or costly route; not optimal	UCS gives the cheapest route
Greedy search	Looks only at distance to goal, ignores risk	UCS includes risk cost